

A Case Study of Developer Bots: Motivations, Perceptions, and Challenges

Sumit Asthana
asumit@umich.edu
University of Michigan
Ann Arbor, USA

Hitesh Sajnani
hitesh.sajnani@thetradedesk.com
Trade Desk
Redmond, Washington

Elena Voyloshnikova
elvoyl@microsoft.com
Microsoft
Redmond, Washington

Birendra Acharya
Birendra.Acharya@microsoft.com
Microsoft
Redmond, Washington

Kim Herzig
kim.herzig@microsoft.com
Microsoft
Redmond, Washington

ABSTRACT

Continuous integration and deployment (CI/CD) is now a widely adopted development model in practice as it reduces the time from ideas to customers. This adoption has also revived the idea of "shifting left" during software development – a practice intended to find and prevent defects early in the software delivery process. To assist with that, engineering systems integrate developer bots in the development workflow to improve developer productivity and help them identify issues early in the software delivery process.

In this paper, we present a case study of developer bots in Microsoft. We identify and analyze 23 developer bots that are deployed across 13,000 repositories and assist about 6,000 developers daily in their CI/CD software development workflows. We classify these bots across five major categories: Config Violation, Security, Data-privacy, Developer Productivity, and Code Quality. By conducting interviews and surveys with bot developers and bot users and by analyzing about half a million historical bot actions spanning over one and a half years, we present software workflows that motivate bot instrumentation, factors impacting their usefulness as perceived by bot users, and challenges associated with their use. Our findings echo existing issues with bots, such as noise, and illustrate new benefits (e.g., cross-team communication) and challenges (e.g., too many bots) for large software teams.

CCS CONCEPTS

• **Human-centered computing** → *Empirical studies in collaborative and social computing; User interface management systems*; • **Software and its engineering** → **Collaboration in software development; Software development process management; Software maintenance tools; Security and privacy** → Software security engineering.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0327-0/23/12...\$15.00

<https://doi.org/10.1145/3611643.3616248>

KEYWORDS

devbots, software quality, software maintenance, developer productivity

ACM Reference Format:

Sumit Asthana, Hitesh Sajnani, Elena Voyloshnikova, Birendra Acharya, and Kim Herzig. 2023. A Case Study of Developer Bots: Motivations, Perceptions, and Challenges. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, December 3–9, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3611643.3616248>

1 INTRODUCTION

Software development has moved towards continuity through methods like Agile [21] and DevOps [13]. The emphasis on continuous delivery, including continuous integration [17] helps teams to react to market changes faster [23]. However, fast delivery in itself is not a success metric as there is little value in having continuous delivery pipelines release erroneous code into production at speed [26]. To prevent this, teams have developed the culture to 'shift left', i.e., 'find and prevent issues early' in the development life cycle: the more issues you find early, the faster it is to fix them and reduce delivery time [22, 41, 47].

Automation is an essential component in shifting left as teams embark on a continuous delivery journey and adopt DevOps culture [40, 41] – and developer support bots play a significant role in the automation strategy to assist developers in their development workflow [25, 49]. Developer bots can be defined as a "conduit or interface between users and services that could be further enhanced through a conversational interface, personalization and memory" [43]. Software developers have been increasingly using developer bots in open source [51] and industry [46] for tasks like fixing bugs, automating testing, refactoring code, and performance analysis [29, 51].

Developing effective developer bots requires improving both their functional aspects (e.g., technology) [29, 34, 37, 46, 55] and user-centered factors like communication, expectations [44] and noise [16, 51]. While existing research highlights both benefits and challenges faced by individual developers across different organizational contexts, we lack an understanding of the challenges large organizations face in effectively deploying and managing multiple developer bots and how multiple bots operating simultaneously impact developer workflows.

To fill this gap, we study 23 developer bots that impact the workflows of more than 6,000 developers across 13,000 repositories in Microsoft. These developer bots are deployed in continuous delivery environments [17] and act during different phases of software development (e.g., work item creation and pull request review) to assist software developers. We use semi-structured interviews with bot developers and users, survey bot users, and analyze about one and a half years of bot usage data (750,000 bot actions). Specifically, our goals are summarized by the three research questions below:

RQ1: What processes and needs motivate developers to instrument bots in a large software engineering organization? What challenges do they face in developing the bots?

RQ2: What is the experience of developers when using various developer bots (benefits and challenges)?

RQ3: How do developers engage with different types of developer bots?

We find that a bot platform provides effective support for data, permissions, and feedback for bots, allowing bot developers to focus on bot logic. We also find broad applicability of bots, such as automating routine workflows (e.g., FAQs), making information more widely visible, and facilitating collaborations. However, with several bots operating in the same context, developers need to prioritize bot recommendations. The urgency of the issue highlighted by the bot, its communication, and its relevance play an integral part in this prioritization. We synthesize these findings across different bot types and situate them in the existing literature to highlight the benefits and challenges of developer bots in large software teams in the industry.

2 RELATED WORK

Better automation capabilities (e.g., through Artificial Intelligence and software frameworks) are leading to the development of smarter bots that are more integrated into people's workflows. Online communities are increasingly employing bots to automate various tasks and offload the work of humans. Bots enforce norms on Wikipedia [19], fight vandalism [38], perform task routing [11], or handle Q&A on Stackoverflow [34]. Social bots in Twitter [2] have also been called "automated social actors" because of aggregating and presenting tweets or actively tweeting themselves.

In software engineering as well, over the years, developers are increasingly using bots in tasks like agile team management [1, 29], pull request management [51], code refactoring [54], automatic dependency management [32], development support environment [24], automated static analysis [10, 12]). To consolidate and facilitate the study of these growing diverse bots, early research proposed a high-level categorization of "Code," "Test," "DevOps," "Documentation," and "Support" bots [43] or categories that capture their intrinsic and environmental properties [36]. Storey et al. [43] define bots as "conduits to automated services" and encompass many bot applications studied on GitHub, such as ensuring license agreements, reporting failures, and running tests [49]. Such use cases have found wide acceptability across practitioners in industry [15]. However, there is an increasing demand for smarter bots that can also offload the burden of routine tasks from developers [15].

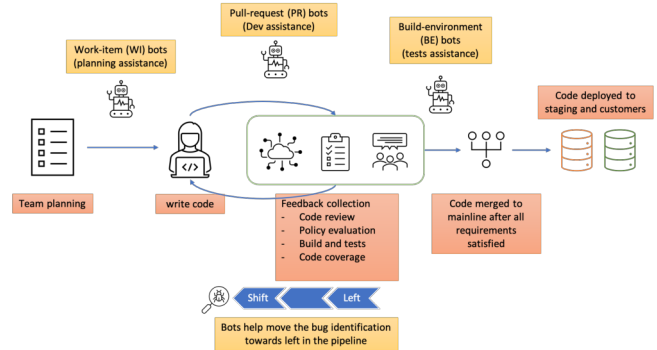


Figure 1: Development pipeline depicting the role of bots in different stages of software development and the ability to "shift left" in detecting bugs earlier than later

Due to most bots currently providing a "support" role, developers have to manage feedback both from their peers and bots, increasing their workload. With several bots operating on the same codebase, research suggests that increasing developer fatigue [34] limits bot usefulness [8]. By surfacing crash reports, welcoming newcomers, and alerting developers of sensitive codebase [18, 49] on public mediums like pull-requests, bots also add a social dimension as the developers' response to bots is publicly visible. Thus, explanations of the bot's actions, clarity of their recommendations, and correctness have all been found to impact perceived usefulness and trust, in addition to their functional characteristics [34]. To manage several different bots, Wessel et al. [50] also proposed the idea of meta-bots that act as a "middleware" between developers and individual bots. This separation provides additional benefits like minimizing bot noise by providing additional controls for bot operations.

However, insights on bots from individuals across organizational contexts do not allow us to understand how bots might work in large software organizations where teams follow established organizational work practices [17, 56] under explicitly structured teams from top to bottom. Knowledge about processes that bots automate in large software infrastructures, complexities arising from integrating actions of several bots, and comparing different types of bots can help inform developer bots that automate tasks with minimal disruption to developers.

3 BACKGROUND AND STUDY CONTEXT

This study was conducted at the US division of Microsoft. The company uses the widely adopted CI/CD pipeline [17] for software development. In this paper, we study a bot ecosystem within the organization that supports developers in different stages of the CI/CD pipeline. Developers and project managers plan projects and tasks on work-item boards (online task planners), resulting in a prioritized backlog. Then, they write code to address the backlog and move the software development process forward. After local testing, developers open up a pull-request (PR) in the development environment and request code review from their peers. Bots support this workflow through numerous automations (e.g., code coverage, static analysis checks, and test suites). Developers receive feedback

from code reviews and the various developer bots in an iterative cycle (e.g., policy checks, build results, code coverage, test runs). Once the developer has addressed all the feedback (from their peers and developer bots), they complete the pull-request. After that, the updated code gets deployed to the staging environment and progressively to different production rings and the customer. The overall goal of this pipeline is to make the development process as efficient as possible while reducing the time from planning to the deployment phase (Figure 1) so that value can be deployed to the customer faster. Thus, developers spend a lot of effort to ensure that bugs and vulnerabilities are detected as early as possible in the pipeline ("shift left").

At the time of the study, the platform has been active in the company for two and a half years, impacting about 6,000 developers (see Table 2). Table 1 describes the key terminologies used in the paper to refer to the different roles of developers in the context of bots and software development.

The bot platform acts as a middleware between the development environment and individual bots. It provides bots access to the development environment (pull requests, work-items, build-environment) and the associated data (e.g., commits, code reviews, comments). Bots can perform different actions (e.g., creating pull-requests, posting comments on pull-requests, editing task descriptions) like any human developer. The bot platform receives data on new software development activity from the development environment (e.g., create/update of pull-requests) and sends it to subscribed bots. The bots use the data to perform an action. This action can be one or more of 1) providing an insight on the pull-request (e.g., "this pull-request adds 10 new library calls"), 2) recommending a change on a pull-request (e.g., "You should not check-in credentials") or 3) completing the task automatically (e.g., automatically merging a pull-request). Figure 2 displays a screenshot of an example security recommendation. We refer to the form of the recommendation (text contents and associated visuals) as "*bot recommendation*". Additionally, developers can reply to the bot recommendation to provide feedback on its usefulness or change the recommendation status from "active" to "fixed" to indicate that they addressed the issue (Figure 2).

Table 1: Key terminologies used in the paper

Term	Description
Repository	Centralized collection of code, documentation and assets for an application's source
Pull-request authors	Developers who create pull-requests
Pull-request reviewers	Developers who provide feedback on pull-requests through code-review
Bot developers	Developers who create and maintain bots
Bot users	Developers whose workflows are impacted by bot actions when they create or review pull-requests/work-items or perform testing
Development environment	CI environment described in Section 3

Table 2: Impact statistics of the bot platform during the study period (January 2021 - August 2022)

PRs	Developers	Repositories	#Recommendations	#Bots
434,669	6,000	13,700	778,500	23

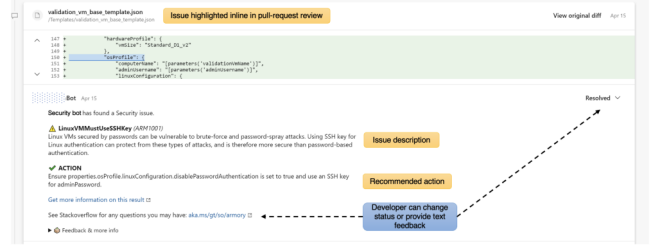


Figure 2: A security bot displaying an inline recommendation on a pull-request and options for developers to provide feedback.

4 STUDY METHODOLOGY

We followed a mixed methods approach to study the usage of bots and their benefits & challenges within the organization. We used semi-structured interviews to understand the benefits and challenges of developing and using bots. We validate our interview findings of bot use through a follow-up survey and supplement our findings with an analysis of historical bot use data. In addition to describing the method in the paper, we provide details of the interviews, interview codes, survey, and log data in supplementary materials [5].

We refer to developers who developed the bots as "bot developers" and developers who use the bots in their software engineering workflows as "bot users" (see Table 1). Both "bot developers" and "bot users" are software developers within Microsoft. Between May 2021 - August 2021, we interviewed 13 bot developers and 9 bot users and administered a follow-up survey to 56 bot users. Table 3 summarizes participant details.

Bot developers We used bot documentations that described their role, bot's data logs, and contact information of their developers to gather information about the bots. Using these documentations, we invited all 21 bot developers for 30-45 minute interviews and 13 accepted our invite. We also invited two bot platform developers for the interview, and both accepted the invite. We asked them about – 1) their motivations for developing the bots, 2) ease or difficulty with bot development, and 3) ease or difficulty with bot maintenance.

Bot users To recruit bot users, we analyzed bot usage logs and identified developers who authored or reviewed at least two pull-requests where bots performed actions prior to the study. Wherever possible, we considered at least three users for each bot type in our recruitment list to ensure our list covered all bots. A total of 21 developers satisfied this criteria, and we invited them for 15-30 minute interviews. Of these, *nine* accepted the invite to discuss their experience with the bots. We asked participants 1) if they are aware of bot recommendations and understand their purpose?, 2) if and when do they find the recommendations useful and their reasons, 3) if and when do they find the recommendations not useful and

their reasons, 4) if they ever gave feedback to the bot and what channels did they use to give feedback.

We conducted semi-structured interviews remotely via video calls, with one research team member leading the interviews and taking notes. Before the start of each session, we obtained informed consent from the participants. We also familiarized participants with the study's objective and explicitly mentioned the voluntary nature of the research. If participants consented, we recorded the interviews; otherwise, one research team member took notes. The study was approved by the institutional review board.

We analyzed the interviews using thematic analysis [7]. The interview data consisted of about 18 hours of recordings and notes taken during the interviews. The lead author went through the interview transcriptions and notes over multiple passes and assigned open codes line by line to the transcripts. For bot developers, the codes captured motivations for developing the bots (e.g., increase test coverage, reduce review time) and challenges in bot development (e.g., unclear metrics for success, no information on engagement). For bot users, the codes captured the developer's opinions about bot use and associated problems (e.g., providing too many suggestions helps find reviewers). The lead author discussed the emerging codes with the study team members over multiple passes and revised the codes, merging overlapping codes and refining the coding to yield a set of final codes. The final codes were then categorized into themes using affinity diagramming [20]. We used Miro board¹ for the affinity diagramming step. We followed the best practices in HCI to ensure reliability when coding and discussing codes [30]. Figure 3 illustrates the codes that emerged from the interviews.

Bot use survey To further refine and validate our insights of the findings from bot user interviews, we used the interview findings to develop a survey of bot users (N = 54). The survey had four parts: asking developers about their 1) awareness of the bots, 2) when and why they find the recommendations useful, 3) when and why they find the recommendations not useful, and 4) if and when they ever provided any feedback to the bot. The four parts of the survey align with the parts of the bot user interviews asking about bot awareness, scenarios when bots were useful, and not useful, and about providing feedback to the bots. Each question was multiple choice, and the answers to the multiple choice questions were populated from the insights we derived from the bot user interviews. We ensured completeness by providing an "other" field for each response and an open-ended "Any other feedback" question at the end of the survey. We analyzed free-text responses similar to our interviews by coding them and grouping them into themes. We administered the survey to about 360 developers who had come across at least one bot recommendation between January 2021 - July 2021 (response rate 15%) when working on their pull-requests. At the beginning of the survey, we provided a screenshot of an example recommendation and a link to the internal wiki containing a description of the bot platform to jog the participants' memory. The example recommendation screenshot was randomly selected for each survey recipient to avoid bias against any specific recommendation type.

¹<https://miro.com/>

Historical data analysis We analyzed the log data of bot use between *January 2021 to August 2022* to support RQ3. To understand bot engagement, we analyzed the developer activity (i.e., replies to the bot's recommendation and changes to bot recommendation status from "active" to "not-active") on different bot recommendations.

Bot developer interviews helped us answer RQ1, bot user interviews and bot user survey helped us answer RQ2, and historical data analysis enabled us to answer RQ3.

Table 3: Participant details, research methodology associated with the participants, and their experience

Participant group	Participant details	Number
Bot platform developers (interview)	Software engineers with average 5 years of experience	P1-2
Bot developers (interview)	Software engineers and managers with 1-12 years of experience	P3-15
Bot users (interview)	Software engineers with 1-12 years of experience	P16-24
Bot users (survey)	Software engineers with 1-12 years of experience	54
Bot users (engagement)	Software engineers across the organization	6,000

5 BOTS MOTIVATIONS AND DEVELOPMENT CHALLENGES (RQ1)

We categorize the bots based on the motivations that emerged from our interviews with bot developers. These motivations highlight the need for specific software teams to automate a particular workflow to save time and increase code quality.

5.1 Bots on the Platform

Table 4 summarizes the list of bots, their purpose, their category, details of where they operate, and the number of recommendations by the bots in the study period. We observed a total of 23 bots on the platform, which were a mix of bots that provided recommendations on pull-requests and bots that performed operations without requiring any developer intervention. We describe the needs and workflows that motivated bot instrumentation in the following sections.

5.2 Bot Developer Motivations

Common platform for accelerated bot development The 23 bots on the platform were on-boarded within a period of 12 months. A common platform with access to the development environment data allowed individual bots to *subscribe to necessary data-stores to run analysis and generate insights* (P02). P01 explained the reason for the fast instrumentation of bots.

During the design of the bot platform, instead of having each bot as an individual app with its own logic, data-store, and interface, we decided to go for an App-store-like model. Our (bot platform) team owns and maintains

Table 4: List of bots hosted on the platform along with their purpose, category, artifact they operate on, and the action expected of the developer in the bot operation context. PR - pull-request, WI - work-item, BE - build environment. "N/A" for #Recs means the bot does not generate recommendations

Name	Purpose	Category	Artifact	ActionExpected	Recs
CloudProtect	Detect vulnerabilities in cloud resource management templates	Security	PR	Remove vulnerable code	30,880
SpamAnalysis	Detect exposed secrets checked-in in PRs	Security	PR	Remove secret	20,816
PrivacyRisk	Block pull-requests if potential privacy guidelines violated	Security	PR	Address privacy risk	1,893
Secret bypass	Alert when PR contains secret and dev has bypassed secret scanning	Security	PR	Remove secret	5,780
ConfigAlert	Services generates a config file during build time. When a PR is published, the newly generated config file is compared with the old config file, and an alert is raised in case of a breaking change	ConfigViolation	PR	update mismatch-ing dependencies	966
Threat Model Quality & SecReview	Designed to automate the process of both evidence collection and validation of the work-items generated from the internal "Data-privacy security assessment" questionnaires	Data-privacy	WI	complete & update security requirements	18,821
Nitpicker	Remind developers of missed dependencies, configs, updates that are routinely followed in the repository	ConfigViolation	PR	update missing code	52,397
Nudge	Remind developers of stale pull requests	DevProductivity	PR	attend to PR	63,109
PR notifications	Summarize evaluation status of branch policies on PRs	CodeQuality	PR	Awareness	
Related file	Remind developers of related file changes based on missing dependencies	DevProductivity	PR	update the suggested file	11,133
Static analysis	Surface results of static analysis tools	CodeQuality	PR	update PR	50,218
Reviewer	Automatically add reviewers to PR	CodeQuality	PR	Review PR	239,636
CodeInsights	Support crash insights, telemetry, experimentation, and testing-based insights. Empower developers to write quality code, fix bugs, and monitor services in production	CodeQuality	PR	Awareness	11,512
Intent	Surface intent (e.g., code, testing, refactor, etc.)	CodeQuality	PR	Awareness	N/A
PR Quantifier	Statistics about the PR (lines, methods, etc.) for awareness during the review	CodeQuality	PR	Awareness	156,517
Flaky test management	Automate handling of flaky tests	DevProductivity	PR	N/A	52,380
MassPR	Automatic PR creations across many repos (e.g., updating a dependency across the codebase)	DevProductivity	BE	N/A	N/A
ICM	Support for incident creation	DevProductivity	WI		111
PR SignOff	Automatically approve PRs and link work items for configured files	DevProductivity	PR	N/A	93,912
CodeAccessibility	Flag UI components that violate accessibility guidelines	CodeQuality	PR	N/A	2,182
TestPassBot	Reviews and approves pull requests related to validated test suites	DevProductivity	PR	N/A	5,002
PRSurvey	Solicit developer feedback on pull-request support by bots	CodeQuality	PR	N/A	3,761

the platform, and the developers create bots (apps) on demand when they see the need for automation.

Another key benefit of the collaborative app-store-like model is *offloading the burden of data collection and storage from bot developers* so that they can focus on *writing code for the bot's functionality* (P02).

M1 ConfigViolation. Developers created these bots to catch configuration issues or violations of standard repository-specific rules that potentially get missed during code reviews and automated testing (P04) but can subsequently cause build failures and regression

issues. For example, catching missed dependencies, incompatible library calls across repositories that testing cannot monitor, or static analysis checks. These bots may be a characteristic of automation needs of large organizations which have a large number of repositories with complex test infrastructures and configurations. P3's team used a configuration violation bot to guide newcomers towards standard team practices.

Our team uses the bot "Nitpicker" to keep developers from missing upgrading the package version when the package is updated

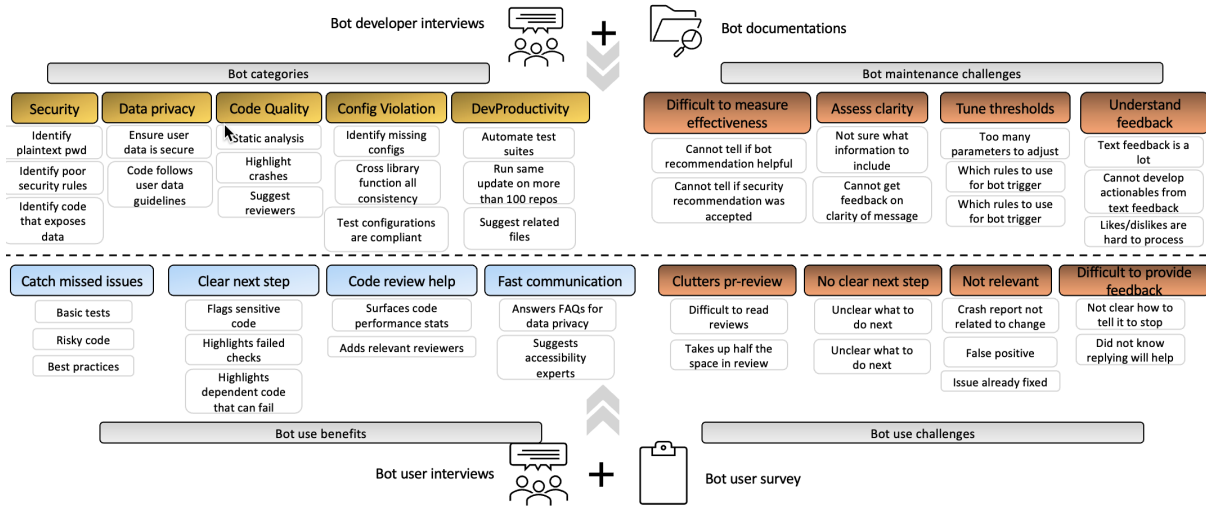


Figure 3: Codes from interviews with bot developers and bot users that emerged in thematic analysis

M2 Security. Teams created security recommendation bots to remind developers of vulnerabilities in the code that can *potentially lead to security exploits* (P09). These security policies can be organization-wide or specific to a group of teams based on their work practices. Security bots recommended severe and, in most cases, urgent organization-wide security flaws and *required developers to address the security recommendations before the pull-request could be completed* (P08). Possible examples include *detecting plaintext passwords to access company resources that developers check-in version control by mistake* (P09) or incorrectly configured security policies for virtual machines.

We built the "CloudProtect" bot to ensure that developers do not create virtual machines that violate standard security practices (P10)

M3 Data-privacy. To better safeguard sensitive user data, the organization has dedicated Data-privacy teams that provide expertise to product development teams on sensitive data practices and help them resolve issues with managing such data. The workflow is similar to customer support for resolving an issue. Before the release of any product, 1) The Data-privacy team assesses code for leakage of sensitive user data, and 2) Failure of these assessments results in a follow-up communication between a Data-privacy team expert and the software product developer until the issue is resolved. Limited Data-privacy team experts assist several product team members, which leads to an availability bottleneck. The majority of the support that Data-privacy experts provide to different developer teams involves answering common repeated queries like frequently asked questions (FAQs). To address this availability bottleneck, the Data-privacy team instrumented the "ThreatModelQuality" and "SecReview" bots. They automatically scan product codes for violations, open work-items for communication, and answer frequent questions (FAQ) on behalf of the privacy experts to provide product developers with fast and easy availability of information. Once the easy and frequently asked queries are resolved, a Data-privacy expert takes over the communication. P11 noted

Using the bot as an asynchronous communication medium has been helping us address security concerns at scale as the bot offloads several routine tasks - like flagging issues and informing resolution.

M4 CodeQuality. These bots nudge developers towards writing better-quality code. They recommend practices that vary from organization-level norms like *enforcing python coding style guides* (P06) to repository-specific guidelines like formatting instructions for configuration files. The recommendations from these bots may sometimes lead to more turnaround time for the pull-request due to the additional work needed to improve code quality. Most bots that we observed in this category were not reported in prior literature [15, 51]. For example, the "CodeAccessibility" bot suggests accessibility guidelines to developers when they make changes to User Interface (UI) components, like using "alt" text in HTML. The developer of the "CodeInsights" bot (P07) noted

We built the bot to reveal interesting insights about the code in the changed files, such as crashes during builds. By surfacing such statistics, we expect more conversations during code reviews on code performance.

M5 DevProductivity. These bots improve developers' productivity by automating some aspects of their development cycle. For example, the "Flaky test bot" runs flaky tests (tests that fail intermittently due to randomness in code) automatically across a set of repositories and reports results. The "CodeReview" bot finds and adds the right reviewers on pull-requests, and the "PRSignOff" bot automatically completes routine pull request changes (for package updates and configuration changes). Instead of generating suggestions on pull-requests, they automatically finish tasks for developers.

Comparison with previous taxonomies. While existing taxonomies provide a baseline to categorize bots [15, 43], they were designed to motivate the study of bots [43] or elicit an individual's understanding of the bots [16, 51]. Thus, we observed that prior taxonomies did not cover all the workflows that motivated

bot instrumentation in our study. Our categorization is grounded in specific dev-ops use cases and overlaps with Erlenhov et al. in uni-directional communication and system-initiated response. CodeQuality and ConfigViolation bots fit the traditional picture of bots as automated conduits to a service [15, 42] ("Alex bot" in Erlenhov et al., "support bots" in Storey et al. [43]). Our categorization also highlights novel use cases like Security and Data-privacy, indicating the need for finer-grained bot classifications. The majority of the bots on the bot platform provided insights or recommendations on pull-requests; therefore, the majority of our insights for RQ2 and RQ3 are based on pull-request bot recommendations.

5.3 Bot Developer Challenges (RQ1)

We now discuss the challenges we uncovered in our interviews with bot developers that they face in developing and maintaining the bots.

Table 5: Challenges for bot maintainers

Dimension	Interview
O1 Difficult to measure effectiveness	8/13
O2 Assessing clarity of bot recommendations	9/13
O3 Tune bot thresholds	2/13
O4 Sensemaking of bot users' feedback	8/13

O1 Difficult to measure effectiveness Most bots in our study provide recommendations to developers, and it is difficult to assess whether developers accepted that recommendation unless they like/dislike or reply to bot recommendations (see Section 3). Developers rarely have the time to provide such additional signals, and even if they do, assessing whether the bot saved the developer's time in software development is difficult because of several confounding factors. Developers of the bot "ThreatModelQuality" mention (P10)

We do not have a way to tell if developers find value in our recommendations. Developers do not reply to each recommendation's feedback template, so we are left with using proxy metrics for assessing value.

O2 Assessing clarity of bot recommendations Many bots that flag issues in pull-requests also provide the next steps to help users resolve the issue (e.g., highlighting lines in code that violate configurations or security policies). Bot developers struggled to improve the clarity of the bot recommendations due to the lack of a structured method to assess the feedback of bot users coming from the bot recommendation channel. For example, security bots use codes to denote security vulnerabilities (e.g., S0203), but security bot developers found that *surfacing such codes to bot users is meaningless if they are not explained* (P06) as non-security developers cannot be expected to possess security know-how. P05 mentions

We are constantly trying to update the instructions that our bot suggests based on the bot user's feedback. We address each feedback one by one, and the process is cumbersome.

O3 Tune bot thresholds Two bots ("CloudProtect" and "CodeInsights") have the functionality to configure the type and threshold of recommendations that they surface to bot users. For example,

the "CodeInsights" bot generates crash reports for library function calls by checking if the number of crash occurrences of a library function call has exceeded a minimum threshold in software builds. "Reviewer" bot that suggests code reviewers uses a *threshold for a minimum number of contributions by a developer on a changed file to recommend them for code review* (P12). These thresholds can vary depending on the bot user's needs. Bot developers found it difficult to set such threshold values and often resorted to heuristics.

We generally show all the recommendations the bot can generate for the subscribing teams. However, we need a good way to assess if certain recommendations make sense in the context of the team or not. (P13)

O4 Sense making of bot users' feedback. Bot users can like or dislike bot recommendation comments, change the bot comment status from "active" to "not-active" or reply to bot recommendation to provide feedback (see Figure 2) that gets stored in bot logs. Bot developers retrieved these logs to *aggregate the like statistics and comments in a dashboard* (P07). While such statistics indicated bot popularity, they did not help bot developers develop actionable insights from the feedback. Maintainers of the bot "CodeInsights" used a dashboard to summarize feedback from different sources, which they reviewed in weekly meetings.

We have a dashboard that summarizes the percentage of developers that like the bot's recommendation and change its status to "fixed." We also look at the list of all bot recommendations where bot users provide feedback on its usefulness. Making sense of all this information is difficult. (P07)

6 BOT USER EXPERIENCE (RQ2)

The findings in this section are derived from interviews and surveys with developers who used the bots ("bot users").

6.1 Bot Use Benefits

The themes that emerged from bot user interviews suggest that developers found bots useful when they 1) helped catch missed issues, 2) were easy to understand, 3) suggested a clear next step, and 4) helped in code review. Bot user survey results support this finding. In response to the general usefulness of the recommendations, pull-request authors (mean = 3.63, and std = 0.97) found the recommendations marginally more useful than pull-request reviewers (mean = 3.52, std = 1). Table 6 summarizes the interview findings and corresponding survey results for reasons on the usefulness of bot recommendations.

We discuss the benefits for bot users in more detail below.

Table 6: Benefits from developer bots (interviews & survey)

Dimension	Interview	Survey
B1 Catches missed issues	6/9	51%
B2 Facilitates faster communication	3/9	29%
B3 Suggests clear next step	2/9	22%
B4 Helps in code review	2/9	16%

B1 Catches missed issues One of the most frequent uses of developer bots in the organization is to remind developers for

checking-in configurations, library calls, and scripts that get missed during development and pull-request review. Missing checking in such code *"leads to build failures later"* (P16), and required developers to *"create a follow-up pull-request to fix the issue"* (P16), increasing their workload. Developers found bot recommendations helpful for *"preventing a failure from a missed code"* (P17) or *"saving on pull-request iterations"* (P16). We identified the bots "Nitpicker," "Static analysis," and "ConfigAlert" for this category.

We configured the Nitpicker bot to generate reminders for configuration files, and it helps newcomers learn the nuances of configurations without burdening a senior developer to point that out. (P17)

B2 Facilitates faster communication. For the bots "ThreatModelQuality", "CodeInsights", "PRQuantifier", and "CodeAccessibility" bot users reported faster and easier communication during code-review. Users of the "ThreatModelQuality" bot reported getting *"quick help on basic privacy questions compared to waiting an entire day when someone had to manually answer them"* (P18). This aligns with the motivation of the privacy review team that instrumented the bot (see M3 - "Data-privacy" for motivation). The "CodeAccessibility" bot surfaced accessibility issues when the user interface (UI) related code was changed in the pull-request, which allowed *"reviewers to discuss those issues faster on the bot comment thread"* (P18). Similarly, the "CodeInsights" bot surfaced performance statistics of the functions that were changed in the pull-request. This enabled code reviewers to suggest improvements to pull-request by simply *"replying to the bot recommendation containing the performance report"* (P19).

B3 Suggests clear next step. Whenever bots highlighted a clear recommended action with the issue (e.g., "Cloud Protect," "Live Secrets Detection," and "ConfigAlert"), bot users could easily take action based on the recommendation. Bots that just highlighted an issue without a clear recommended next step (e.g., "CodeInsights," "PR notifications") made it more likely for developers to *ignore the recommendations* (P17) because of time and attention constraints during pull-request review.

The "Live Secrets Detection" bot does a good job of telling me a password is present in the checked-in code and highlights the password as well. I simply go ahead and remove the password in the next pull-request iteration. (P18)

B4 Helps in code review. Bots also facilitated code-review by finding the right developers for code-review ("Reviewer") or providing newcomers style guidelines and common practices for the repository when changing code (P21) ("Nitpicker"). Displaying overall pull-request statistics ("PRQuantifier") like functions added, removed, modified, number of new library calls, and static analysis violations ("Static Analysis") were generally perceived by developers as providing an additional *automatic code review on the pull-request* (P23) as long as the recommendations were easy to understand.

Developers also saw a cultural value in such recommendations. Bots take the burden off the code-reviewers to nitpick every minor issue in the pull-request, letting code-reviewers provide feedback on the more important aspects of the review.

I think that "SpamAnalysis" provides value, more of the cultural sort - i.e., we told the pusher adding credentials

in PR was a bad idea, and this bot confirms it was indeed a bad idea." (P24)

6.2 Bot Use Challenges

The themes from the interviews of "bot users" suggest that developers find bot recommendations challenging when they 1) clutter the pull-request review interface, 2) provide no clear next step, 3) are not perceived as relevant by developers, 4) are difficult to give feedback to. Table 7 summarizes these findings from interviews and corresponding survey results.

Table 7: Challenges faced by developer bot users (interviews & survey)

Dimension	Interviews	Survey
D1 Clutters the pull-request review interface	5/9	21%
D2 No clear next step	5/9	21%
D3 Not relevant to the change at hand	4/9	42%
D4 Unclear/Difficult to provide feedback	8/9	14%

D1 Recommendation takes up a lot of space. Some bot recommendations ("CodeInsights," "StaticAnalysis") provided detailed information that would take up a significant portion of the screen on the pull-request review interface. This cluttered the pull-request review screen that developers use to *review comments left by peer developers during code review* (P18). Analyzing these bots' logs, we identified that such bots provided detailed information to the bot users, like full logs of crash reports or long descriptions of security vulnerabilities that *could have been provided on an external page linked to the bot's recommendation message* (P20).

D2 No clear next step. Bot recommendations varied in the kind of actions that they suggested depending on their purpose. "ConfigViolation" and "Security" bots stated the exact steps to address the issue. The "CodeInsight" category of bot recommendations were open-ended, providing statistics about code runtime performance. The absence of clear next steps left users uncertain about the recommendation.

I see the recommendations, and I am aware of those issues. It is not immediately clear to me what I should do with that recommendation, and hence, I mostly ignore it. (P21)

D3 Not relevant to the change at hand Bot recommendations that only contained a piece of information or analysis about the changed code but no explanation for the same were regarded as irrelevant by bot users.

When I see these recommendations, I see a lot of statistics and crash reports. I am not able to connect the dots as to how this would be useful for me.

D4 Unclear/Difficult to provide feedback Developers can reply to bot recommendation comments on the pull-request review interface (see Section 3) and it gets logged as feedback for bot developers (described in Figure 2). A high percentage of users expressed their *unawareness* (P21) about using the "reply" feature to provide feedback to the bot developers (see Figure 4 from the survey).

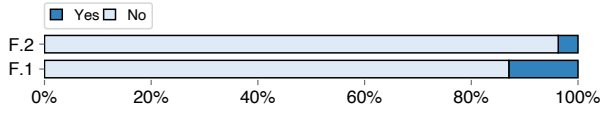


Figure 4: Survey: Reasons for not replying to bot comment
F.1 - Wasn't aware of the reply feature
F.2 - Wasn't aware that replying was of any value
 A large percentage of bot users were either unaware of providing feedback or did not consider it as a practice.

I was unaware that if I replied to the bot interface, the feedback would be acted upon or looked at. Every comment contains the "reply" text box, and I believe it did not hold any significance for comments authored by bots. (P21)

To sum up, bot users indicated several challenges stemming from the mismatch in priorities between bot developers and bot users. For example, developers of "DevProductivity" and "CodeQuality" bots intended to alert users of best coding practices and performance issues and notify them of delays. However, these may sometimes conflict with the bot user's desire to finish the pull-request on a deadline. "Recommendation takes up space" and "No clear next step" have been echoed in previous work in open source as well (e.g., "Communication breakdown" in Wessel et al. [52], "design challenges" in Brown et al. [8]). We also diverge from prior studies in aspects like "adoption challenges," which we did not come across because dedicated teams develop the bots within the organization for addressing critical software engineering needs. We also study the effect of several bots working together and observe that too many bots can cause noise, even if individual bots are helpful.

7 ENGAGEMENT (RQ3)

Table 8: Statistics about bots that we use to study engagement

Category	#Recs	#Repos	Bots
CodeQuality	220,429	2,580	CodeAccessibility, StaticAnalysis, CodeInsights, PRQuantifier
ConfigViolation	81,308	219	ConfigAlert, ConfigBot, Nitpicker
DevProductivity	79,244	3,974	Testpassbot, ReltedFiles, Nudge
Security	40,016	4,391	SecretScanByPassPolicy, SpamAnalysis, CloudProtect, LiveSecretsDetection, Data-privacyRisk

To validate our qualitative findings about bots, we analyzed the logs of bot user interactions with bot recommendations. Specifically, across bot categories, we examined 1) the percentage of bot recommendations where bot users change the recommendation status from "active" to one of "fixed", "closed", or "won't-fix" and 2) Number of bot user replies to bot recommendations. Developers changing the bot recommendation status from "active" to one of

"fixed", "closed", or "won't-fix" serves as a proxy for their interaction with the bot recommendations. A status of "fixed" indicates that the pull-request author has addressed the comment, "closed" indicates that the discussion is closed, and "won't-fix" indicates that the author will not address the issue in the current pull-request.

Table 8 describes the statistics of bots that generated recommendations for developers during the study period. Figure 5a shows the final status of bot recommendation as "active" and "not-active" ("fixed", "closed", or "won't-fix") when the pull-request was completed, for repositories across the bot categories "CodeQuality", "ConfigViolation", "DevProductivity", and "Security."

To quantify developer engagement, we examined the unique number of developers who reply to the bot recommendations. We calculate the "Mean developer replies per bot category" which represents the average number of replies received by bots in a given category within a repository. Figure 5b displays the distribution of "Mean developer replies per bot category" across all repositories. Bot categories with higher engagement should exhibit a higher overall mean number of replies for repositories. We find that "CodeQuality" bots, on average, receive a higher number of replies on their recommendations compared to "ConfigViolation" and "Security" bots. To test if this effect is significant, we conducted one-way ANOVAs. Because our data was not normally distributed, we performed Align Rank Transform (ART) [53] before running the ANOVAs and performed post-hoc pairwise analysis using ART-c [14] with Holm-Bonferroni correction. We found that bot type has a significant effect on "Mean developer engagement" on bot comments ($F_{3,4545} = 332.08, p < .0001$). The post hoc test found a significant difference ($p < 0.0001$) between all categories. Again, "ConfigViolation" ($median = 0.03, mean = 0.13$) had the lowest developer replies, while "CodeQuality" ($median = 1.0, mean = 0.87$), "Security" ($median = 1.0, mean = 0.60$) and "DevProductivity" ($median = 1.0, mean = 1.11$) were comparable.

Higher overall engagement for "CodeQuality" bots aligns with our interview findings. "Security" and "ConfigViolation" bots surface urgent issues in pull-requests which are actionable and fixed by pull-request authors relatively quickly. "CodeQuality" bots show higher engagement as they highlight issues related to quality and runtime performance that requires developers to have more in-depth conversations to improve the quality (see M4 CodeQuality in Section 5).

8 DISCUSSION

Our findings support prior research on developer bots' integration in software ecosystems [29, 50] and shed new light on how developers engage with such bots.

8.1 Where Existing Bots Provide Value

Automating simple routine workflows Many bots in our study (e.g., "Data-privacy", "ConfigViolation" category of bots) automate parts of software development processes like providing answers to FAQs and reminding new developers of common practices in the team. While FAQ-style bots are gaining popularity in social Q&A websites (e.g., Stackoverflow [55]), they have seen limited use in the software engineering landscape [34]. Bots that automate such common workflows free up developer's time to focus on more

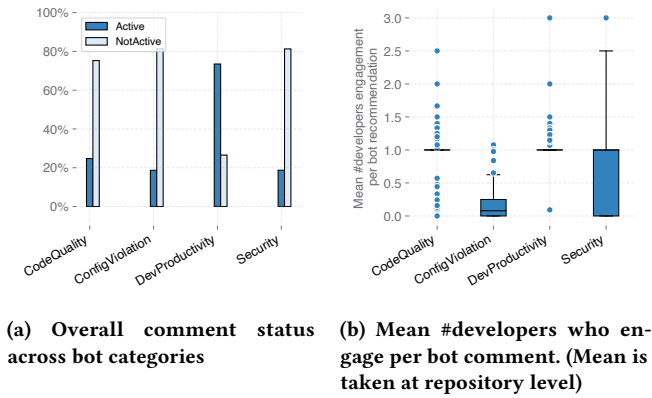


Figure 5: Engagement statistics of bot recommendations.

complex aspects of the project. Erlenhov et al. found such use cases with "Alex" style bots [15].

Communication and Collaboration Software development activities like maintenance and support require knowledge distributed among developers, and communication over chat, emails, and mailing lists enables such knowledge sharing [35]. Yet due to the abundance of information sources, developers are unable to keep up with the channels, find difficulty in managing multiple information sources, and newcomers find it challenging to find the right information [44]. Bots can address such issues by providing information scaffolding to support developers and facilitate communication across teams by simplifying domain-specific terms for outsiders. For example, the Nitpicker bot provided additional communication to newcomers and guided them through repository best practices [4], and the Security bot faced an adoption challenge because of obscure security rules.

Difficult for humans, easy for bots As found in prior work [15], bots can improve human capabilities by providing quick search into issues (B1), answering frequent questions (M3), and reporting crashes and status of services. Such tasks are difficult for humans. By automating the tedious task of search and retrieval, developer bots can provide humans with a summary of issues that developers can act on [52, 55].

8.2 Where Bots Need to Improve

Bot platform as meta bot We study a bot platform that hosts 23 bots and acts as a meta bot [50] that manages other bots. From interviews with two bot platform developers, we find that this concept is effective in accelerating bot development because several bots share infrastructure that allows individual bot developers to focus on developing the bot logic. Centralized control of bots' access to different repositories also simplifies enabling bots for new repositories, and disabling them for existing repositories if they do not provide value.

Relevance and information overload Developers have to manage multiple sources of information from Continuous integration systems [17], such as build statuses, pull-request statuses, and deployment updates [44]. Developer bots help streamline these workflows by consolidating the multiple sources of information [9],

but it still leaves developers to make sense of the abundant information. Our finding of unbalanced workload allocation by the "Reviewer" bot and too many notifications from the "CodeInsights" bot echoes findings on the "Mention Bot" [37] on GitHub that tags developers in relevant pull-requests and was found to generate too many redundant notifications. Wessel et al. [52] also reported bot recommendation noise as a substantial challenge limiting bot adoption on GitHub. Additionally, our quantitative analysis of historical bot logs in Section 7 suggests that despite the noise, bot engagement is high for bots that report urgent or severe issues. Thus, future bot ecosystems should consider priority and relevance of bot recommendations based on developer needs, as well as the organization-wide severity of the issue.

Actionability and low false positives We observed low actionability as one of the challenges in bot use (see "D2" in Section 6.2). Developers have to consider information from multiple sources in the pull-request review interface (e.g., comments from peer developers, test results, bot recommendations). Developer bot recommendations need to be precise and consider the developer's background, cognitive availability, and fatigue when making recommendations [15, 31, 48, 51]. For example, under bot user challenges (D4), we found that even if bots provide a channel to seek feedback, very few developers actually provide feedback on bot actions. Possible reasons could be the lack of an interactive interface that is intuitive for feedback and developers' limited attention when addressing numerous pull-request issues.

Feedback and communication Feedback from bots is an important component of Human-Machine interaction that has been shown to help users understand the situation better and make appropriate decisions [3]. With increasing conversational capability, developer bots can move from simple "conduits" to services [43] to more interactive contexts that answer developer queries about its use and help them make sense of it. Prior research [8, 52] has found a "communication breakdown" between bots and developers, and we find similar evidence where developers were unclear on how to provide the bot feedback (see D4 in Section 6.2). Developer bots should present clear feedback about their actions and also provide acknowledgment to users to ensure continued trust and feedback.

Flighting and experimentation Setting up a new developer bot requires several expensive steps – securely collecting software development data, developing the algorithm to run the analysis, and maintaining the bot. Bot discovery and bot adoption is another challenge [52] that bot developers face. Our interviews with bot developers reveal that a bot platform allows easy data sharing between multiple bots for generating insights and a fast turnaround time for new analysis. This has the potential to accelerate the experimentation and deployment of developer bots [6] in a more realistic context, lower the barrier to entry, and provide easy maintenance.

Explainability of bot actions Most of the bots in our study provide insights on the pull-requests or suggest actions that developers can take to fix issues when making changes to the code. Bots with ambiguous and difficult-to-understand recommendations received less engagement than bots that were more precise and suggested clear next steps [33]. For effective bots, *their purpose, what they can and can't do must be evident, and match user expectations* [27]. Bots

in our study provided text-based recommendations, and our findings align with prior findings on the difficulty of communication in text-based feedback to developers [8, 28].

9 THREATS TO VALIDITY

We situate our work in Microsoft's development environment. We study bots operating in the popular Continuous Integration (CI) environment [17], making our findings broadly applicable, but they still have certain limitations.

Scope and external validity We provide a case study of a developer bot ecosystem in the context of the large organization Microsoft. The organizational processes described in Section 3 follows a widely adopted industry standard [41], and the bot interactions like generating recommendations on pull-requests and automating tests are extensively studied in prior work in other organizations [37]. Our results are applicable in scenarios that use continuous integration systems and instrument bots to assist developers in the pull-request review interface. However, our study does not cover all possible types of bots. We did not observe intelligent bots in our study that use Machine Learning, which could bring in challenges with probabilistic predictions [16, 52] ("Sam" bots in Erlenhov et al.). Our proposed bot categories emerged from observing organizational workflows that motivated the bots there may be other workflows that our categorization does not cover.

Information saturation We conducted our interviews during the initial phases of bot platform development and recruited participants until we reached saturation in the information gathered or ran out of participants [45]. While the final user base of the bot platform at the end of the two-year study period was 6,000 developers, the number of developers using the bot platform at the time of survey administration was roughly 1,000, and only 400 of these active users had seen bot actions more than once. Of these, we sent the survey to 360 developers who most frequently used the bot recommendations and received responses from 54 developers (response rate ~15%). Thus, we had a limited set of participants to recruit for interviews and the survey. Future work can widen the scope of bot study through a broader participant pool and across different organizations.

Diversity of bots Our study aimed to provide an account of the bot ecosystem within the organization. Therefore, our results align with a higher-level overview of bot engagement across categories but do not delve deeper into the usefulness and challenges of individual bots. Prior work has studied individual bots in depth [26, 33, 37], but more studies are needed to understand newer types of bots (e.g., configuration bots, rule enforcing bots) and their impact on development workflow.

Quantitative analysis of historical bot engagement We study bot engagement by measuring developer actions on bot recommendations – 1) changing the status from "active" to "fixed" and providing text replies to bot recommendations like "useful." While this is a reasonable proxy as most developers perform one of the above actions on popular collaboration platforms like GitHub [39], developers may still miss engaging with the recommendations when pressed for time or when the pull-request is populated with multiple bot recommendations, while still acting on the recommendation suggested by the bot. Controlled longitudinal studies are

required to study the long impact of bots on issues found in the codebase and developer productivity.

10 CONCLUSION AND FUTURE WORK

Our work addresses the questions related to practical bot usage in CI/CD environments. Through a mixed-methods approach, we build an understanding of bot motivations, understand the benefits and challenges of bot users, and complement it with large-scale engagement data to substantiate our findings. We highlight the non-functional aspects of bots that affect their usefulness in the development context. Different types of bots receiving different levels of engagement from developers are a consequence of the growing workload of developers and the prioritization of tasks based on the urgency of the issue.

Our findings reinforce the consideration of human factors when building bots beyond their intrinsic usefulness because developer bots are part of a socio-technical ecosystem where human factors play a crucial role in the usefulness of technical artifacts [18]. Future work can leverage these insights to build bots that align better with developers to maximize their utility.

11 DATA AVAILABILITY

We provide interview and survey questionnaires, codes that emerged in interview analysis, survey results, and replication for quantitative engagement in supplementary materials [5]. The README file in the supplementary package provides a complete description of the supplementary materials. We are unable to provide full interview transcripts due to the restrictions placed on the work due to industry applications.

REFERENCES

- [1] Ruth Ablett, Ehud Sharlin, Frank Maurer, Jorg Denzinger, and Craig Schock. 2007. BuildBot: Robotic Monitoring of Agile Software Development Teams. In *RO-MAN 2007 - The 16th IEEE International Symposium on Robot and Human Interactive Communication*. 931–936. <https://doi.org/10.1109/ROMAN.2007.4415217>
- [2] Norah Abokhodair, Daisy Yoo, and David W. McDonald. 2015. Dissecting a Social Botnet: Growth, Content and Influence in Twitter. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing* (Vancouver, BC, Canada) (CSCW '15). Association for Computing Machinery, New York, NY, USA, 839–851. <https://doi.org/10.1145/2675133.2675208>
- [3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fournay, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300233>
- [4] Maryl Arciniegas-Mendez, Alexey Zagalsky, Margaret-Anne Storey, and Allyson Fiona Hadwin. 2017. Using the Model of Regulation to Understand Software Development Collaboration Practices and Tool Support. In *Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing* (Portland, Oregon, USA) (CSCW '17). Association for Computing Machinery, New York, NY, USA, 1049–1065. <https://doi.org/10.1145/2998181.2998360>
- [5] Sumit Asthana. 2023. A case study of developer bots artifacts. <https://doi.org/10.6084/m9.figshare.19382123.v4>. [Online; accessed 28-August-2023].
- [6] Ivan Beschastnikh, Mircea F. Lungu, and Yanyan Zhuang. 2017. Accelerating Software Engineering Research Adoption with Analysis Bots. In *2017 IEEE/ACM 39th International Conference on Software Engineering: New Ideas and Emerging Technologies Results Track (ICSE-NIER)*. 35–38. <https://doi.org/10.1109/ICSE-NIER.2017.17>
- [7] Virginia Braun and Victoria Clarke. 2012. *Thematic analysis*. American Psychological Association. <https://doi.org/10.1037/13620-004>
- [8] Chris Brown and Chris Parnin. 2020. Sorry to Bother You Again: Developer Recommendation Choice Architectures for Designing Effective Bots. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (Seoul, Republic of Korea) (ICSEW'20). Association for Computing Machinery, New York, NY, USA, 56–60. <https://doi.org/10.1145/3387940.3391506>

- [9] Fabio Calefato and Filippo Lanubile. 2016. A Hub-and-Spoke Model for Tool Integration in Distributed Development. In *2016 IEEE 11th International Conference on Global Software Engineering (ICGSE)*. 129–133. <https://doi.org/10.1109/ICGSE.2016.12>
- [10] Antônio Carvalho, Welder Luz, Diego Marcílio, Rodrigo Bonifácio, Gustavo Pinto, and Edna Dias Canedo. 2020. C-3PR: A Bot for Fixing Static Analysis Violations via Pull Requests. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 161–171. <https://doi.org/10.1109/SANER48275.2020.9054842>
- [11] Dan Cosley, Dan Frankowski, Loren Terveen, and John Riedl. 2007. Suggest-Bot: Using Intelligent Task Routing to Help People Find Work in Wikipedia. In *Proceedings of the 12th International Conference on Intelligent User Interfaces* (Honolulu, Hawaii, USA) (IUI '07). Association for Computing Machinery, New York, NY, USA, 32–41. <https://doi.org/10.1145/1216295.1216309>
- [12] Diego Costa, Cor-Paul Bezemer, Philipp Leitner, and Artur Andrzejak. 2021. What's Wrong with My Benchmark Results? Studying Bad Practices in JMh Benchmarks. *IEEE Transactions on Software Engineering* 47, 7 (2021), 1452–1467. <https://doi.org/10.1109/TSE.2019.2925345>
- [13] Christof Ebert, Gorka Gallardo, Josune Hernantes, and Nicolas Serrano. 2016. DevOps. *Ieee Software* 33, 3 (2016), 94–100. <https://doi.org/10.1109/MS.2016.68>
- [14] Lisa A. Elkin, Matthew Kay, James J. Higgins, and Jacob O. Wobbrock. 2021. An Aligned Rank Transform Procedure for Multifactor Contrast Tests. In *Proceedings of the 34th Annual Symposium on User Interface Software and Technology* (Virtual Event, USA) (UIST '21). Association for Computing Machinery, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3472749.3474784>
- [15] Linda Erlenhov, Francisco Gomes de Oliveira Neto, Riccardo Scandariato, and Philipp Leitner. 2019. Current and Future Bots in Software Development. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. 7–11. <https://doi.org/10.1109/BotSE.2019.00009>
- [16] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. 2020. An Empirical Study of Bots in Software Development: Characteristics and Challenges from a Practitioner's Perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) (ESEC/FSE 2020). Association for Computing Machinery, New York, NY, USA, 445–455. <https://doi.org/10.1145/3368089.3409680>
- [17] M. Fowler and M. Foemmel. 2005. Continuous integration, <http://www.martinfowler.com/articles/continuousIntegration.html>.
- [18] R. Stuart Geiger. 2009. The Social Roles of Bots and Assisted Editing Programs. In *Proceedings of the 5th International Symposium on Wikis and Open Collaboration* (Orlando, Florida) (WikiSym '09). Association for Computing Machinery, New York, NY, USA, Article 30, 2 pages. <https://doi.org/10.1145/1641309.1641351>
- [19] R. Stuart Geiger. 2013. Are Computers Merely "Supporting" Cooperative Work: Towards an Ethnography of Bot Development. In *Proceedings of the 2013 Conference on Computer Supported Cooperative Work Companion* (San Antonio, Texas, USA) (CSCW '13). Association for Computing Machinery, New York, NY, USA, 51–56. <https://doi.org/10.1145/2441955.2441970>
- [20] Gunnar Harboe and Elaine M. Huang. 2015. Real-World Affinity Diagramming Practices: Bridging the Paper-Digital Gap. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI '15). Association for Computing Machinery, New York, NY, USA, 95–104. <https://doi.org/10.1145/2702123.2702561>
- [21] Orit Hazzan and Yael Dubinsky. 2009. *Agile software engineering*. Springer Science & Business Media. <https://doi.org/10.1007/978-1-84800-198-5>
- [22] Michael Hilton, Nicholas Nelson, Timothy Tunnell, Darko Marinov, and Danny Dig. 2017. Trade-Offs in Continuous Integration: Assurance, Security, and Flexibility. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (Paderborn, Germany) (ESEC/FSE 2017). Association for Computing Machinery, New York, NY, USA, 197–207. <https://doi.org/10.1145/3106237.3106270>
- [23] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering* (Singapore, Singapore) (ASE '16). Association for Computing Machinery, New York, NY, USA, 426–437. <https://doi.org/10.1145/2970276.2970358>
- [24] G.E. Kaiser, P.H. Feiler, and S.S. Popovich. 1988. Intelligent assistance for software development and maintenance. *IEEE Software* 5, 3 (1988), 40–49. <https://doi.org/10.1109/52.2023>
- [25] Noureddine Kerzazi and Bram Adams. 2016. Botched Releases: Do We Need to Roll Back? Empirical Study on a Commercial Web App. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. 574–583. <https://doi.org/10.1109/SANER.2016.114>
- [26] Jasmine Latendresse, Rabe Abdalkareem, Diego Elias Costa, and Emad Shihab. 2021. How Effective is Continuous Integration in Indicating Single-Statement Bugs?. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 500–504. <https://doi.org/10.1109/MSR52588.2021.00062>
- [27] Carlene Lebeuf, Alexey Zagalsky, Matthieu Foucault, and Margaret-Anne Storey. 2019. Defining and Classifying Software Bots: A Faceted Taxonomy. In *Proceedings of the 1st International Workshop on Bots in Software Engineering* (Montreal, Quebec, Canada) (BotSE '19). IEEE Press, 1–6. <https://doi.org/10.1109/BotSE.2019.00008>
- [28] Dongyu Liu, Micah J. Smith, and Kalyan Veeramachaneni. 2020. Understanding User-Bot Interactions for Small-Scale Automation in Open-Source Development. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '20). Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/3334480.3382998>
- [29] Christoph Matthies, Franziska Dobrigkeit, and Guenter Hesse. 2019. An Additional Set of (Automated) Eyes: Chatbots for Agile Retrospectives. In *Proceedings of the 1st International Workshop on Bots in Software Engineering* (Montreal, Quebec, Canada) (BotSE '19). IEEE Press, 34–37. <https://doi.org/10.1109/BotSE.2019.00017>
- [30] Nora McDonald, Sarita Schoenebeck, and Andrea Forte. 2019. Reliability and Inter-Rater Reliability in Qualitative Research: Norms and Guidelines for CSCW and HCI Practice. *Proc. ACM Hum.-Comput. Interact.* 3, CSCW, Article 72 (nov 2019), 23 pages. <https://doi.org/10.1145/3359174>
- [31] Christopher Mendez, Hema Susmita Padala, Zoe Steine-Hanson, Claudia Hilderbrand, Amber Horvath, Charles Hill, Logan Simpson, Nupoor Patil, Anita Sarma, and Margaret Burnett. 2018. Open Source Barriers to Entry, Revisited: A Sociotechnical Perspective. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 1004–1015. <https://doi.org/10.1145/3180155.3180241>
- [32] Samim Mirhosseini and Chris Parnin. 2017. Can automated pull requests encourage software developers to upgrade out-of-date dependencies?. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 84–94. <https://doi.org/10.1109/ASE.2017.815621>
- [33] Martin Monperrus. 2019. Explainable Software Bot Contributions: Case Study of Automated Bug Fixes. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. 12–15. <https://doi.org/10.1109/BotSE.2019.00010>
- [34] Alessandro Murgia, Daan Janssens, Serge Demeyer, and Bogdan Vasilescu. 2016. Among the Machines: Human-Bot Interaction on Social Q&A Websites. In *Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems* (San Jose, California, USA) (CHI EA '16). Association for Computing Machinery, New York, NY, USA, 1272–1279. <https://doi.org/10.1145/2851581.2892311>
- [35] Peter Naur. 1985. Programming as theory building. *Microprocessing and microprogramming* 15, 5 (1985), 253–261. [https://doi.org/10.1016/0165-6074\(85\)90032-8](https://doi.org/10.1016/0165-6074(85)90032-8)
- [36] Elaha Paikari and André van der Hoek. 2018. A Framework for Understanding Chatbots and Their Future. In *Proceedings of the 11th International Workshop on Cooperative and Human Aspects of Software Engineering* (Gothenburg, Sweden) (CHASE '18). Association for Computing Machinery, New York, NY, USA, 13–16. <https://doi.org/10.1145/3195836.3195859>
- [37] Zhenhui Peng, Jeehoon Yoo, Meng Xia, Sunghun Kim, and Xiaojuan Ma. 2018. Exploring How Software Developers Work with Mention Bot in GitHub. In *Proceedings of the Sixth International Symposium of Chinese CHI* (Montreal, QC, Canada) (ChineseCHI '18). Association for Computing Machinery, New York, NY, USA, 152–155. <https://doi.org/10.1145/3202667.3202694>
- [38] Martin Potthast, Benno Stein, and Robert Gerling. 2008. Automatic vandalism detection in Wikipedia. In *European conference on information retrieval*. Springer, 663–668. https://doi.org/10.1007/978-3-540-78646-7_75
- [39] Samaneh Saadat, Natalia Colmenares, and Gita Sukthankar. 2021. Do Bots Modify the Workflow of GitHub Teams?. In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*. 1–5. <https://doi.org/10.1109/BotSE52550.2021.00008>
- [40] Andreas Schaefer, Marc Reichenbach, and Dietmar Fey. 2013. Continuous integration and automation for DevOps. In *IAENG Transactions on Engineering Technologies*. Springer, 345–358. https://doi.org/10.1007/978-94-007-4786-9_28
- [41] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2017. Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access* 5 (2017), 3909–3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- [42] Klaas-Jan Stol, Paul Ralph, and Brian Fitzgerald. 2016. Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) (ICSE '16). Association for Computing Machinery, New York, NY, USA, 120–131. <https://doi.org/10.1145/2884781.2884833>
- [43] Margaret-Anne Storey and Alexey Zagalsky. 2016. Disrupting Developer Productivity One Bot at a Time. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Seattle, WA, USA) (FSE 2016). Association for Computing Machinery, New York, NY, USA, 928–931. <https://doi.org/10.1145/2950290.2983989>
- [44] Margaret-Anne Storey, Alexey Zagalsky, Fernando Figueira Filho, Leif Singer, and Daniel M. German. 2017. How Social and Communication Channels Shape and Challenge a Participatory Culture in Software Development. *IEEE Transactions on Software Engineering* 43, 2 (2017), 185–204. <https://doi.org/10.1109/TSE.2016.2584053>
- [45] Anselm Strauss and Juliet M Corbin. 1997. *Grounded theory in practice*. Sage.

- [46] Simon Urli, Zhongxing Yu, Lionel Seinturier, and Martin Monperrus. 2018. How to Design a Program Repair Bot? Insights from the Repairator Project. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice* (Gothenburg, Sweden) (ICSE-SEIP '18). Association for Computing Machinery, New York, NY, USA, 95–104. <https://doi.org/10.1145/3183519.3183540>
- [47] Bogdan Vasilescu, Yue Yu, Huaimin Wang, Premkumar Devanbu, and Vladimir Filkov. 2015. Quality and Productivity Outcomes Relating to Continuous Integration in GitHub. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (Bergamo, Italy) (ESEC/FSE 2015). Association for Computing Machinery, New York, NY, USA, 805–816. <https://doi.org/10.1145/2786805.2786850>
- [48] Mihaela Vorvoreanu, Lingyi Zhang, Yun-Han Huang, Claudia Hilderbrand, Zoe Steine-Hanson, and Margaret Burnett. 2019. From Gender Biases to Gender-Inclusive Design: An Empirical Investigation. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3290605.3300283>
- [49] Mairieli Wessel, Bruno Mendes de Souza, Igor Steinmacher, Igor S. Wiese, Ivanilton Polato, Ana Paula Chaves, and Marco A. Gerosa. 2018. The Power of Bots: Characterizing and Understanding Bots in OSS Projects. *Proc. ACM Hum.-Comput. Interact.* 2, CSCW, Article 182 (Nov. 2018), 19 pages. <https://doi.org/10.1145/3274451>
- [50] Mairieli Wessel and Igor Steinmacher. 2020. The Inconvenient Side of Software Bots on Pull Requests. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (Seoul, Republic of Korea) (ICSEW'20). Association for Computing Machinery, New York, NY, USA, 51–55. <https://doi.org/10.1145/3387940.3391504>
- [51] Mairieli Wessel, Igor Steinmacher, Igor Wiese, and Marco A. Gerosa. 2019. Should I Stale or Should I Close? An Analysis of a Bot That Closes Abandoned Issues and Pull Requests. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. 38–42. <https://doi.org/10.1109/BotSE.2019.00018>
- [52] Mairieli Wessel, Igor Wiese, Igor Steinmacher, and Marco Aurelio Gerosa. 2021. Don't Disturb Me: Challenges of Interacting with Software Bots on Open Source Software Projects. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW2, Article 301 (oct 2021), 21 pages. <https://doi.org/10.1145/3476042>
- [53] Jacob O. Wobbrock, Leah Findlater, Darren Gergle, and James J. Higgins. 2011. The Aligned Rank Transform for Nonparametric Factorial Analyses Using Only Anova Procedures. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, BC, Canada) (CHI '11). Association for Computing Machinery, New York, NY, USA, 143–146. <https://doi.org/10.1145/1978942.1978963>
- [54] Marvin Wyrich and Justus Bogner. 2019. Towards an Autonomous Bot for Automatic Source Code Refactoring. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. 24–28. <https://doi.org/10.1109/BotSE.2019.00015>
- [55] Bowen Xu, Zhenchang Xing, Xin Xia, and David Lo. 2017. AnswerBot: Automated generation of answer summary to developers' technical questions. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 706–716. <https://doi.org/10.1109/ASE.2017.8115681>
- [56] Andy Zaidman, Bart Van Rompaey, Arie Van Deursen, and Serge Demeyer. 2011. Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining. *Empirical Software Engineering* 16 (2011), 325–364. <https://doi.org/10.1007/s10664-010-9143-7>

Received 2023-02-02; accepted 2023-07-27